1 Rappresentazione dell'Informazione e Logica Booleana

1.1 Rappresentazione dell'Informazione

1.1.1 Sistemi di numerazione

Un numero può essere rappresentato in diverse **basi numeriche**. La base b determina il numero di simboli disponibili per rappresentare le cifre.

- **Base 2 (binaria)**: cifre $\{0, 1\}$.
- **Base 8 (ottale)**: cifre $\{0, 1, \ldots, 7\}$.
- **Base 10 (decimale)**: cifre $\{0, 1, \ldots, 9\}$.
- **Base 16 (esadecimale)**: cifre $\{0, 1, \ldots, 9, A, B, C, D, E, F\}$.

Un numero N in base b si scrive come:

$$N = a_{n-1}b^{n-1} + a_{n-2}b^{n-2} + ... + a_1 b + a_0$$

dove a_i è la cifra nella posizione i ($0 \leq a_i < b$).

Esempio:

$$(1011)_2 = 1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0 = (11)_{10}$$

Quante cifre servono in base b? Con n cifre in base b si possono rappresentare b^n combinazioni diverse. Se vogliamo rappresentare N informazioni distinte, il numero minimo di cifre richiesto è:

$$n \geq \lceil \log_b(N) \rceil$$

Esempi:

- Per rappresentare 100 simboli diversi in base 2: $n \geq \lceil \log_2(100) \rceil = \lceil 6.64 \rceil = 7$. Servono quindi almeno 7 bit.

- Per rappresentare 500 simboli diversi in base 16: $n \geq \lceil \log_{16}(500) \rceil = \lceil 2.7 \rceil = 3$. Servono quindi 3 cifre esadecimali.

1.1.2 Numeri Naturali

Conversioni tra basi

Da decimale a base b Si usa il metodo delle **divisioni successive**.

1. Si divide il numero N per la base b.
2. Si prosegue con le divisioni finchè non arrivo a quoziente 0.
3. Si raccolgono i resti (r) dalle divisioni successive in ordine inverso rispetto a come sono stati ottenuti.
4. La sequenza di resti in base b rappresenterà il numero N nella codifica cercata.

Esempio: Convertire 45_{10} in binario:

	2
45	1
22	0
11	1
5	1
2	0
1	1
0	

I resti presi in ordine inverso rispetto a quello con cui vengono ottenuti danno come risultato: $45_{10} = 101101_2$.

Da base b a decimale Si usa il metodo della **somma di prodotti**. Se un numero in base b è

$$(a_n a_{n-1} ... a_1 a_0)_b$$

dove ogni cifra a_i è compresa tra 0 e $b-1$, allora il valore decimale si calcola come

$$N_{10} = a_n \cdot b^n + a_{n-1} \cdot b^{n-1} + ... + a_1 \cdot b^1 + a_0 \cdot b^0.$$

Si andrà quindi a moltiplicare ogni cifra per una potenza della base corrispondente alla sua posizione sommando infine tutti i risultati ottenuti.

Esempio: $(253)_8 = 2 \cdot 8^2 + 5 \cdot 8^1 + 3 \cdot 8^0 = 171_{10}$

Conversioni tra basi potenze di 2 Entrambi i procedimenti esposti hanno validità generale e permettono la conversione da una qualsiasi base b_i a b_j. Solitamente, il procedimento delle divisioni successive si utilizza per il passaggio da base 10 a una qualsiasi base b. Differentemente, il procedimento della somma di prodotti risulta immediato per la conversione da una qualsiasi base b a base 10. Nel caso di conversioni in cui la base 10 non sia coinvolta è consuetudine effettuare un doppio passaggio, da una base generica a base 10 e da base 10 alla base desiderata. Questo passaggio intermedio attraverso la base 10 può essere omesso, quando si effettua una conversione di base tra due basi tali che una è una potenza dell'altra. Passare dalla base 2 (binario) alla base 8 (ottale) o 16 (esadecimale) è particolarmente semplice, perché $8 = 2^3$ e $16 = 2^4$. Questo significa che ogni cifra ottale corrisponde esattamente a un gruppo di 3 bit, e ogni cifra esadecimale corrisponde a un gruppo di 4 bit.

1. Si parte dalla cifra meno significativa (a destra).
2. Si raggruppano i bit a blocchi di 3 (per l'ottale) o 4 (per l'esadecimale).
3. Se l'ultimo gruppo (quello più a sinistra) non è completo, si aggiungono zeri all'inizio per completarlo.
4. Si converte ciascun gruppo nella cifra corrispondente.

Esempio 1: da binario a ottale Convertire $(101101)_2$ in base 8.

$$(101101)_2 = (101)(101)_2$$

Ogni gruppo di 3 bit corrisponde a una cifra ottale: $101_2 = 5_8$, $101_2 = 5_8$.

$$(101101)_2 = (55)_8$$

Esempio 2: da ottale a binario Convertire $(253)_8$ in base 2.

$$2 = 010_2, \quad 5 = 101_2, \quad 3 = 011_2$$

$$(253)_8 = (010101011)_2$$

1.1.3 Aritmetica Binaria

Somma

L'operazione di somma è determinata da due elementi principali: il bit di **risultato** ed il bit di **resto** definiti come segue:

risultato

+	0	1
0	0	1
1	1	0

riporto

+	0	1
0	0	0
1	0	1

Genericamente, dati due operandi di n bit, $A = [a_{n-1}...a_1a_0]$ e $B = [b_{n-1}...b_1b_0]$ definiamo la somma come:

$$\begin{array}{cccccccc} c_n & c_{n-1} & \cdots & c_3 & c_2 & c_1 & & \\ & a_{n-1} & \cdots & a_3 & a_2 & a_1 & a_0 & + \\ & b_{n-1} & \cdots & b_3 & b_2 & b_1 & b_0 & = \\ \hline & s_{n-1} & \cdots & s_3 & s_2 & s_1 & s_0 & \end{array}$$

Si noti come, nel caso in cui il bit di riporto $c_n = 1$, non è possibile rappresentare il risultato in n bit. Questo fenomeno viene definito come **overflow**.

Overflow Due casi possibili dopo somma:

1. Il risultato è rappresentabile con il numero di bit a disposizione, $c_n = 0$.
2. Il risultato non è rappresentabile con il numero di bit a disposizione, $c_n = 1$.

Sottrazione

L'operazione di sottrazione è determinata da due elementi principali: il bit di **risultato** ed il bit di **prestito** definiti come segue:

risultato

−	0	1
0	0	1
1	1	0

prestito

−	0	1
0	0	0
1	1	0

Genericamente, dati due operandi di n bit, $A = [a_{n-1}...a_1a_0]$ e $B = [b_{n-1}...b_1b_0]$ definiamo la sottrazione come:

$$\begin{array}{cccccccc} p_n & p_{n-1} & \cdots & p_3 & p_2 & p_1 & & \\ & a_{n-1} & \cdots & a_3 & a_2 & a_1 & a_0 & - \\ & b_{n-1} & \cdots & b_3 & b_2 & b_1 & b_0 & = \\ \hline & s_{n-1} & \cdots & s_3 & s_2 & s_1 & s_0 & \end{array}$$

Analogamente alla somma, in caso il bit di prestito $p_n = 1$ si verifica una situazione anomala definita come **underflow**.

1.1.4 Numeri Relativi

I numeri relativi (in base 2) possono essere codificati utlizzando diverse rappresentazioni:

1. Rappresentazione in modulo e segno.
2. Notazione in complemento a 2

Modulo e Segno

La rappresentazione in modulo e segno si basa sull'utilizzo di due elementi, **segno** e **valore**. In base 2 si utilizza:

- 0 per indicare il segno positivo.
- 1 per indicare in segno negativo.

Il bit di segno occupa la posizione più significativa. Ovvero, dato un elemento $N = a_{n-1}a_{n-2}...a_1a_0$, il bit a_{n-1} rappresenterà il segno, i restanti il valore assoluto.

Esempio: Convertire -14_{10} in binario modulo e segno:

$$\begin{array}{r|l} & 2 \\ \hline 14 & 0 \\ 7 & 1 \\ 3 & 1 \\ 1 & 1 \\ 0 & \end{array}$$

Ricordandoci di aggiungere il bit di segno il risultato sarà: $-14_{10} = 11110_{2\text{MS}}$

Utilizzando questo tipo di rappresentazione per convetire un numero N avremo bisogno di $n = \lceil \log_b(|N| + 1) + 1 \rceil$ bit. Ponendoci in binario (base 2) l'insieme dei valori rappresentabili è $[-(2^{n-1} - 1), 2^{n-1} - 1]$.

Complemento a 2

Il complemento a 2 consente di rappresentare numeri negativi in modo compatto, evitando di fornire due rappresentazioni diverse per lo zero e di conseguenza aumentando anche

anche i valori rappresentabili. Con n bit (in base 2) si possono infatti rappresentare i numeri nell'intervallo:

$$[-2^{n-1},\ 2^{n-1}-1]$$

I domini di rappresentabilità possono essere definiti come:

- Se $N \geq 0$, $n = \lceil \log_b(|N|+1)+1 \rceil$
- Se $N < 0$, $n = \lceil \log_b(|N|)+1 \rceil$

Per i numeri positivi si utilizza la stessa metodologia di conversione utilizzata per i numeri naturali, adattata al numero di bit corretto. Per i numeri negativi si effettuano i seguenti passaggi:

1. Scrivere il numero, in valore assoluto, in binario con n bit.
2. Invertire tutti i bit (complemento a 1).
3. Aggiungere 1.

Esempio: Convertire -12_{10} in binario complemento a 2: Per rappresentare il valore serviranno $n = \lceil \log_2(|-12|)+1 \rceil = 5$ bit.

	2
12	0
6	0
3	1
1	1
0	

$|N| = 12_{10} = 01100_{2C2}$

1. Inverto tutti i bit $\rightarrow$ 10011 (complemento a 1)
2. Sommo 1 $\rightarrow$ 10100 (complemento a 2)

Il risultato finale è $N = 10100_{2C2}$

Somma e Overflow Le somme (e sottrazioni) in complemento a 2 si eseguono normalmente in binario. In alcuni casi, l'eventuale riporto non comporta una condizione di overflow e può essere ignorato mantenendo il risultato finale corretto.

Esempio: $5 + (-5)$ su 8 bit:

$$00000101 + 11111011 = 1\ 00000000$$

Ignoro il riporto: $00000000 = 0$

In generale, dati due operandi a e b distinguiamo:

- $a > 0$, $b > 0$ $a + b < 0 \rightarrow$ Overflow!
- $a > 0$, $b < 0$ $a + b \rightarrow$ Possiamo scartare il bit di carry (se necessario).
- $a < 0$, $b < 0$ $a + b > 0 \rightarrow$ Overflow!

1.1.5 Numeri Razionali

Virgola fissa

La rappresentazione a virgola fissa riserva:

- n bit per la parte intera,
- k bit per la parte frazionaria.

Per effettuare la conversione di un numero frazionario in binario:

- La parte intera si ottiene con il metodo delle divisioni successive.
- La parte frazionaria si ottiene con il metodo delle **moltiplicazioni successive**.

Si moltiplica la parte frazionaria per 2 prendendo la parte intera (che sarà 0 o 1). Si rieffettua la stessa operazione sulla nuova parte frazionaria ottenuta dalla moltiplicazione. Il processo si ripete finchè non si raggiunge il numero massimo di bit riservati per la parte frazionaria o quando la parte frazionaria è 0.

Esempio: Convertire 0.375_{10} in base 2 utilizzando la virgola fissa.

$0.375 \cdot 2 = 0.75 \rightarrow 0$
$0.75 \cdot 2 = 1.5 \rightarrow 1$
$0.5 \cdot 2 = 1.0 \rightarrow 1$

Quindi $0.375 = 0.011_2$.

Virgola mobile (IEEE 754)

La virgola mobile ci permette di "spostare" la virgola a seconda del valore che vogliamo codificare. Il processo si divide in tre fasi: determinazione del **segno**, calcolo dell'**esponente** traslato e calcolo della **mantissa**.

Utilizziamo come esempio il numero $N = +23.75$

Fase del segno (S): il primo bit è dedicato al segno, 0 se positivo 1 se negativo. Quindi, $S = 0$.

Fase di normalizzazione e mantissa (M): Il numero decimale (ignorando il segno) deve essere convertito in binario e poi scritto in notazione scientifica binaria normalizzata.

- Si converte il numero decimale in binario come per la virgola fissa:

  Parte intera $23_{10} = 10111$

  Parte frazionaria 0.75

  $0.75 \cdot 2 = 1.5 \rightarrow 1$
  $0.5 \cdot 2 = 1.0 \rightarrow 1$

  $23.75_{10} = 10111.11_2$.

- Normalizzazione: si sposta la virgola binaria fino a quando non si trova dopo il primo 1. Il numero di posizione in cui la virgola è stata spostata definisce l'**esponente effettivo**. Spostare la virgola significa moltiplicare piuttosto che dividere per una potenza di 2.

  $E_{eff} = 4$.

  Otteniamo così la mantissa implicita 1.011111

- La mantissa M memorizzata è solo la parte frazionaria che segue il bit implicito (bit prima della virgola).

  $M = 01111100000000000000000$ (23 bit)

Fase dell'esponente traslato (E): Dato l'esponente effettivo possiamo ricavare l'esponente come:

$$E = E_{eff} + \text{Bias}$$
$$E = 4 + 127 = 131$$

Si converte infine l'esponente in base 2:

$$131_{10} = 100000011_2$$

Si conclude assemblando le 3 parti ricavate:

$$+23.75_{10} = 01000001101111100000000000000000_2$$

Nota: L'esempio proposto segue lo standard IEEE 754 a singola precisone. Lo standard è caratterizzato da:

- Singola precisione, $E = 8$ bit e $M = 23$ bit
- Doppia precisione, $E = 11$ bit e $M = 52$ bit

Esempio: -13.25_{10}:

$$13.25 = 1101.01_2 = 1.10101_2 \cdot 2^3$$

$$S = 1, \quad E = 3 + 127 = 130 = 10000010_2$$

$$M = 10101000000000000000000$$

Risultato (32 bit):

1 10000010 10101000000000000000000

1.2 Logica Booleana

Un'espressione booleana descrive una condizione, ovvero un'istruzione che, quando valutata, risulta sempre in un valore pari a zero (0, falso) o uno (1, vero), determinata dagli operatori e dal valore delle costanti o variabili in essa contenute. Per comporre espressioni booleane complesse si utilizzano i seguenti operatori:

Operatori relazionali valutano relazioni binarie tra i due operandi:

- `<` minore di
- `<=` minore di o uguale uguale a
- `>` maggiore di
- `>=` maggiore di o uguale a
- `==` uguale a
- `!=` non uguale a (diverso)

Operatori booleani valutano condizioni di verità tra i due operandi

- `&&` AND (congiunzione logica)
- `||` OR (disgiunzione logica)
- `!` NOT (negazione logica)

N.B. Differentemente dalle operazioni AND e OR il NOT è un operatore unario, valutato quindi su singolo input.

Le espressioni logiche rispettano un ordine tra i propri operatori. In assenza di parentesi l'ordine è determinato come (elementi a priorità maggiore in alto):

1. negazione (NOT) `!`
2. operatori di relazione `<`, `>`, `<=`, `>=`
3. uguaglianza `==`, disuguaglianza `!=`,
4. congiunzione (AND) `&&`
5. disgiunzione (OR) `||`

Proprietà: gli operatori `&&` (AND) e `||` (OR) sono commutativi:

- a `&&` b == b `&&` a

- a || b == b || a

Le doppie negazioni si elidono:

- !!a == a

1.2.1 Tabelle di Verità

Le tabelle di verità sono uno strumento logico utilizzato per determinare la validità di espressioni o proposizioni logiche. Esse mostrano tutte le possibili combinazioni di valori di verità (vero o falso) delle variabili coinvolte, permettendo di valutare l'output di un'espressione logica in ciascun caso. La tabella è strutturata come segue (esempio in Tabella 1.1):

- Una riga per ogni possibile assegnamento di valori logici alle variabili
- Una colonna per ogni espressione che compone l'espressione data (inclusa la formula stessa)

Congiunzione (AND)		
A	**B**	**A && B**
0	0	0
0	1	0
1	0	0
1	1	1

Tabella 1.1: Tabella di verità per l'espressione logica A && B

Leggi di De Morgan: illustrano come distribuire la negazione rispetto a || e &&:

- !(a && b) == !a || !b
- !(a || b) == !a && !b

Esempio:

```
!((a >= 5) && (a <= 10))
!(a >= 5) || !(a <= 10)    [De Morgan]
!!(a < 5) || !!(a > 10)    [trasformazione disuguaglianze]
(a < 5) || (a > 10)    [doppia negazione]
```

Utilizzando De Morgan rendiamo l'espressione più leggibile. Avremo quindi in questo caso risultato vero se il valore di a è minore di 5 oppure maggiore di 10.

Esercizio 1.1

Convertire in base 10 la sequenza 101101_2

Esercizio 1.2

Convertire in base 2 il valore 543_{10}

Esercizio 1.3

Convertire il numero binario $(10111001101)_2$ in base 8 (ottale).

Esercizio 1.4

Convertire il numero binario $(11011001011)_2$ in base 16 (esadecimale).

Esercizio 1.5

Eseguire la somma binaria $A = (11011000)_2$ e $B = (01101101)_2$. Assumendo una rappresentazione a 8 bit per numeri naturali, verificare se si verifica un overflow.

Esercizio 1.6

Convertire in base 2 i numeri 77 e 156. Sommare in base 2 i risultati ottenuti.

Esercizio 1.7

Eseguire la sottrazione binaria $A - B$, dove $A = (00110101)_2$ e $B = (01110000)_2$. Assumendo una rappresentazione a 8 bit, determinare il risultato e spiegare se si verifica una condizione di **underflow**.

Esercizio 1.8

Utilizzando 8 bit, rappresentare il numero decimale -105_{10} in Modulo e Segno (MS).

Esercizio 1.9

(a) Si dica qual è l'intervallo di valori interi rappresentabile con la codifica in complemento a due a 9 bit.

(b) Con riferimento a tale codifica indicare, giustificando brevemente le risposte, quali delle seguenti operazioni possono essere effettuate correttamente:

- $-254 - 255$

- $+254 - 253$
- $-18 + 236$
- $+217 + 182$

(c) Mostrare in dettaglio come avviene il calcolo delle operazioni (i) e (ii), evidenziando il bit di riporto e il bit di overflow così ottenuti.

Esercizio 1.10

Convertire il numero decimale $+10.625_{10}$ in binario a virgola fissa, utilizzando 5 bit per la parte intera (incluso il segno) e 4 bit per la parte frazionaria.

Esercizio 1.11

Convertire il numero decimale -0.125_{10} nel formato a virgola mobile IEEE 754 a Singola Precisione (32 bit).

Esercizio 1.12

Si consideri la seguente espressione booleana:

$$(A \;\&\&\; B) \;||\; !C$$

e se ne compili la corrispondete tabella di verità.

Esercizio 1.13

Si consideri la seguente espressione booleana:

$$((!A \;||\; B) \;\&\&\; (A \;\&\&!B)) \;||\; C$$

e se ne compili la corrispondente tabella di verità (ossia una tabella in cui 0 rappresenta il valore logico FALSO, 1 il valore VERO).

Si consideri ora la condizione, scritta in linguaggio C, in cui x e y siano due variabili intere:

$$((!(x < 0)||(y > 0)) \;\&\&\; ((x < 0) \;\&\&\; !(y > 0)))) \;||\; (x > 3)$$

ottenuta dall'espressione booleana sopra indicata sostituendo la variabile A con $x < 0$, la variabile B con $y > 0$, la variabile C con $x > 3$ Si risponda alle seguenti domande giustificando la risposta:

1. L'espressione è vera o falsa quando $x = 5$ e $y = 7$?
2. Se $x < 0$, per quali valori di y l'espressione è vera?

Esercizio 1.14

1. Scrivere la tabella di veritá della proposizione $A \Rightarrow B$ (implicazione logica), sapendo che essa è equivalente alla proposizione $!A \ || \ B$ (due proposizioni sono equivalenti se la loro tabella di verità è identica).

2. Scrivere la tabella di verità della proposizione $A \Leftrightarrow B$ (co-implicazione logica), sapendo che essa è equivalente alla proposizione $A \Rightarrow B \ \&\& \ B \Rightarrow A$.

3. Mostrare che la proposizione $A \Rightarrow B$ non è equivalente alla proposizione $A \ || \ !B$.

Esercizio 1.15

Data l'espressione logica !(A || !B) && C:

1. Utilizzare le Leggi di De Morgan per semplificare la negazione della parentesi.

2. Costruire la tabella di verità completa per l'espressione originale.

Soluzioni

Soluzione dell'esercizio 1.2

$$\begin{aligned} 101101_2 &= 1 \cdot 2^5 + 0 \cdot 2^4 + 1 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 \\ &= 32 + 0 + 8 + 4 + 0 + 1 \\ &= 45 \end{aligned}$$

Soluzione dell'esercizio 1.2

Per convertire il valore 543_{10} saranno necessari $n \geq \lceil \log_2(543) \rceil = \lceil 9.08 \rceil = 10$ bit.

	2
543	1
271	1
135	1
67	1
33	1
16	0
8	0
4	0
2	0
1	1
0	

Il risultato finale è $543_{10} = 1000011111_2$

Soluzione dell'esercizio 1.3

Si raggruppano i bit a blocchi di tre, partendo da destra. Si aggiungono zeri a sinistra se necessario (in questo caso, un solo zero):

$$(\underbrace{010}_{2}\ \underbrace{111}_{7}\ \underbrace{001}_{1}\ \underbrace{101}_{5})_2$$

Si converte ciascun gruppo di tre bit nel suo equivalente decimale (che è anche la cifra ottale):

$$\begin{aligned} 010_2 &= 2 \\ 111_2 &= 4 + 2 + 1 = 7 \\ 001_2 &= 1 \\ 101_2 &= 4 + 1 = 5 \end{aligned}$$

Il risultato si ottiene assemblando le cifre ottali:

$$(10111001101)_2 = (2715)_8$$

Soluzione dell'esercizio 1.4

Si raggruppano i bit in blocchi di 4, partendo da destra, aggiungendo zeri a sinistra per completare l'ultimo blocco:

$$(11011001011)_2 \rightarrow (\underbrace{0110}_{6}\ \underbrace{1100}_{C}\ \underbrace{1011}_{B})_2$$
$$(11011001011)_2 = (6CB)_{16}$$

Soluzione dell'esercizio 1.5

Si esegue la somma binaria:

	1	1	1	1	1	0	0	0	
	1	1	0	1	1	0	0	0	+
	0	1	1	0	1	1	0	1	=
(1)	0	1	0	0	0	1	0	1	

Il risultato è $S = (101000101)_2$.

Poiché la rappresentazione è a 8 bit e il bit di riporto finale (c_9) è **1**, si verifica un **overflow**.

Soluzione dell'esercizio 1.6

Per convertire il valore 77_{10} saranno necessari $n \geq \lceil \log_2(77) \rceil = \lceil 6.26 \rceil = 7$ bit.

	2
77	1
38	0
19	1
9	1
4	0
2	0
1	1
0	

$77_{10} = 1001101_2$

Per convertire il valore 156_{10} saranno necessari $n \geq \lceil \log_2(156) \rceil = \lceil 7.28 \rceil = 8$ bit.

	2
156	0
78	0
39	1
19	1
9	1
4	0
2	0
1	1
0	

$156_{10} = 10011100_2$

Eseguiamo ora la somma binaria:

0	0	1	1	1	1	0	0	
0	1	0	0	1	1	0	1	+
1	0	0	1	1	1	0	0	=
1	1	1	0	1	0	0	1	

Il risultato è $S = (11101001)_2$.

Soluzione dell'esercizio 1.7

1	1	0	0	0	0	0		
0	0	1	1	0	1	0	1	−
0	1	1	1	0	0	0	0	=
1	1	0	0	0	1	0	1	

Il settimo bit necessita di un prestito per completare l'operazione. Il prestito verrà richiesto all'ottavo che essendo 0 non può soddisfare, quindi bisognerà "scalare" ancora più a sinistra, ma non abbiamo più alcun bit disponibile. Questo significa che il risultato non può essere rappresentato come un numero positivo a 8 bit: siamo andati in underflow.

Il risultato binario $(11001001)_2$ è un valore errato in questo contesto.

Soluzione dell'esercizio 1.8

Il numero è negativo, quindi il bit di segno sarà 1. Calcoliamo con i restanti 7 bit il valore assoluto, $|-105| = 105_{10}$:

	2
105	1
52	0
26	0
13	1
6	0
3	1
1	1
0	

$105_{10} = 1101001_2$

Unendo il bit di segno al valore assoluto convertito in base due ottienamo come risultato finale $-105_{10} = 11101001_{2\text{MS}}$

Soluzione dell'esercizio 1.9

(a) I valori rappresentabili vanno da $-2^{m-1} = -256$ a $2^{m-1} - 1 = +255$ compresi.

(b) Le soluzioni delle operazioni sono:

- $-254 - 255$ NO si ottiene un valore negativo troppo grande in valore assoluto
- $+254 - 253$ SI si ottiene un valore piccolo in valore assoluto
- $-18 + 236$ SI si ottiene un valore positivo, grande in valore assoluto ma nei limiti
- $+217 + 182$ NO si ottiene un valore positivo troppo grande in valore assoluto

(c)

- In complemento a 2 a $m = 9$ bit, $(-254)_{10} = (100000010)_{CP2}$ (perchè $(254)_{10} = (011111110)_2$). Da questo possiamo ricavare che $(-255)_{10} = (100000001)_{CP2}$ essendo $-255 = -254 - 1$. La somma diventa:

		(1)								
		1	0	0	0	0	0	0	1	0	+
		1	0	0	0	0	0	0	0	1	=
[1]	(1)	0	0	0	0	0	0	0	1	1	

- 254 in complemento a due è $(011111110)_{CP2} = (011111110)_2$ essendo positivo. Invece $-253 = -254 + 1 = (100000011)_{CP2}$, quindi:

		(1)	(1)	(1)	(1)	(1)	(1)	(1)	(1)		
		0	1	1	1	1	1	1	1	0	+
		1	0	0	0	0	0	0	1	1	=
[0]	(1)	0	0	0	0	0	0	0	0	1	

ignorando il riporto abbiamo il risultato esatto $(000000001)_{CP2} = (1)_{10}$.

Soluzione dell'esercizio 1.10

1. Parte Intera (10_{10} su 5 bit):
$$10_{10} = 1010_2$$
Adattato a 5 bit (segno 0): 01010.

2. Parte Frazionaria (0.625_{10} su 4 bit): Si usa il metodo delle moltiplicazioni successive:
$$\begin{aligned} 0.625 \times 2 = 1.25 &\quad \to 1 \\ 0.25 \times 2 = 0.50 &\quad \to 0 \\ 0.50 \times 2 = 1.00 &\quad \to 1 \end{aligned}$$
Parte frazionaria: $.101_2$. Adattato a 4 bit: 1010.

3. Unione delle due parti: $+10.625_{10} = 01010.1010_2$

Soluzione dell'esercizio 1.11

1. Segno S: Negativo, $S = 1$

2. Conversione Binaria e Normalizzazione (+0.125):
$$\begin{aligned} 0.125_{10} &= 0.001_2 \\ \text{Normalizzazione:} \quad 1.0 &\times 2^{-3} \\ E_{eff} &= -3. \end{aligned}$$

3. Esponente Traslato (E su 8 bit): Bias = 127.
$$\begin{aligned} E &= E_{eff} + 127 = -3 + 127 = 124_{10} \\ E &= 01111100_2 \end{aligned}$$

4. Mantissa (M) su 23 bit: Dalla forma 1.0, la parte frazionaria è 0.

$$M = 00000000000000000000000$$

5. Uniamo $S \mid E \mid M$: $-0.125_{10} = 1 \quad 01111100 \quad 00000000000000000000000$

Soluzione dell'esercizio 1.12

A	B	C	A && B	!C	(A && B) || !C
0	0	0	0	1	1
0	0	1	0	0	0
0	1	0	0	1	1
0	1	1	0	0	0
1	0	0	0	1	1
1	0	1	0	0	0
1	1	0	1	1	1
1	1	1	1	0	1

Soluzione dell'esercizio 1.13

A	B	C	!A	!B	(!A || B)	(A && !B))	((!A || B) && (A && !B))	TOT
0	0	0	1	1	1	0	0	0
0	0	1	1	1	1	0	0	1
0	1	0	1	0	1	0	0	0
0	1	1	1	0	1	0	0	1
1	0	0	0	1	0	1	0	0
1	0	1	0	1	0	1	0	1
1	1	0	0	0	1	0	0	0
1	1	1	0	0	1	0	0	1

Quando $x = 5$, la variabile $C = 1$, quindi l'espressione è vera. Se $x < 0$, la variabile $C = 0$, quindi l'espressione è falsa per qualsiasi valore di y.

Soluzione dell'esercizio 1.14

1. Tabella di veritá di $A \Rightarrow B$:

A	B	!A	!A || B
0	0	1	1
0	1	1	1
1	0	0	0
1	1	0	1

2. Tabella di veritá di $A \Leftrightarrow B$:

A	B	A ⇒ B	B ⇒ A	A ⇒ B && B ⇒ A
0	0	1	1	1
0	1	1	0	0
1	0	0	1	0
1	1	1	1	1

3. Confrontiamo le due tabelle di veritá:

A	B	!B	A || !B	A ⇒ B
0	0	1	1	1
0	1	0	0	1
1	0	1	1	0
1	1	0	1	1

Le due tabelle di veritá non sono identiche, per cui le proposizioni non sono equivalenti.

Soluzione dell'esercizio 1.15

1. Semplificazione (Leggi di De Morgan):

$$\begin{aligned} \text{!(A || !B) \&\& C} &\equiv \text{(!A \&\& !!B) \&\& C} \quad \text{[De Morgan]} \\ &\equiv !A \&\& B \&\& C \quad \text{[Doppia Negazione]} \end{aligned}$$

2. Tabella di verità:

A	B	C	!B	A || !B	!(A || !B) && C
0	0	0	1	1	0
0	0	1	1	1	0
0	1	0	0	0	0
0	1	1	0	0	1
1	0	0	1	1	0
1	0	1	1	1	0
1	1	0	0	1	0
1	1	1	0	1	0